

VITYARTHI – PROJECT



Project Title : Super Calculator

Course Title : Programming in Java

Course Code : CSE2006

Course Type : Flipped course

Course Credits : 3

Professor : Dr. Rizwan Ur Rahman

Slot : F11 + F12

Submitted By : Dilip S

Registration Number : 25BA110590

Submission Date : 10/09/2026

VITYARTHI – PROJECT

INTRODUCTION :

The **SuperCalculator** is a Java-based console application designed to provide students and professionals with a versatile computational tool. Built as part of the VITyarthi initiative, this project demonstrates the practical application of core Java concepts such as control structures, methods, exception handling, and user input/output.

The calculator integrates multiple mathematical modules into a single program, offering:

- **Scientific Functions** (logarithmic, trigonometric, exponential, hyperbolic operations)
- **Matrix Operations** (addition and multiplication)
- **Equation Solvers** (quadratic and linear systems of 2 variables)
- **Statistical Analysis** (mean, median, standard deviation)
- **Utility Functions** (power calculation, prime number check, factorial computation)

By combining these features, the project showcases how Java can be used to build **modular, user-friendly, and robust applications**. Exception handling ensures smooth execution even when users provide invalid inputs, making the program reliable in real-world scenarios.

This project not only strengthens understanding of Java programming fundamentals but also highlights the importance of **structured problem-solving, algorithm design, and interactive user interfaces**. It serves as a practical learning milestone for applying theoretical knowledge to functional software development.

VITYARTHI – PROJECT

PROBLEM STATEMENT :

Engineering students, developers, and analysts often need to perform diverse mathematical operations ranging from scientific functions and matrix manipulations to solving equations, statistical analysis, and number theory.

Traditionally, these tasks require multiple tools: scientific calculators, spreadsheets, or specialized software. Switching between them is inefficient, error-prone, and time-consuming.

PROJECT OBJECTIVES :

1. Design a modular Java program with separate functions for each mathematical category.
2. Provide a clear, menu-driven interface for easy navigation between features.
3. Implement robust error handling to manage invalid inputs gracefully.
4. Ensure accurate calculations across diverse domains (scientific, algebraic, statistical, number theory).
5. Create an extensible framework that allows new features to be added easily.
6. Deliver formatted, user-friendly outputs for clarity in academic and professional use.

VITYARTHI – PROJECT

FUNCTIONAL REQUIREMENTS :

1. User Interaction :

- Display a menu-driven interface with numbered options.
- Accept user input for selecting operations.
- Provide clear prompts for entering values.
- Allow repeated use until the user chooses to exit.

2. The SuperCalculator supports 8 distinct mathematical domains :

(i)Scientific Functions:

Logarithmic, exponential, trigonometric, inverse trigonometric, and hyperbolic functions.

(ii)Matrix Operations:

Addition and multiplication of square matrices.

(iii)Quadratic Solver:

Finds real and complex roots of quadratic equations.

(iv)Linear Solver (2×2):

Solves systems of two linear equations using determinant-based methods.

VITYARTHI – PROJECT

(v)Statistics:

Mean, median, and standard deviation for datasets.

(vi)Power Calculations:

Computes exponents efficiently.

(vii)Prime Number Utilities:

Checks primality.

(viii)Factorial:

Computes factorial values for integers.

3. Exception Handling :

- Handle invalid inputs gracefully (e.g., non-numeric entries).
- Prevent crashes by catching runtime errors.
- Display user-friendly error messages and allow retry.

VITYARTHI – PROJECT

NON-FUNCTIONAL REQUIREMENTS :

1. Performance :

- Executes computations and renders output in under 50 milliseconds per transaction.
- Minimal JVM startup overhead and near-zero memory footprint during interactive sessions.

2. Security :

- Completely local execution without external network telemetry, outbound requests, or tracking.
- Safe parsing of string inputs to prevent buffer overruns or unhandled exception termination.

3. Usability :

- Minimalist syntax footprint with clear prompt guides (>) and contextual error notifications.
- Support for natural input syntax reduces rigid command-line memorization.

4. Reliability & Availability :

- Runs entirely offline without relying on network bandwidth or remote web services.
- Defensive parsing ensures invalid commands do not terminate active REPL sessions.

VITYARTHI – PROJECT

5. Maintainability :

- Modular static method architecture (Process, normalize, getFactor, getGroup, runTemp).
- Implementation of modern Java syntax features (arrow switch expressions) for cleaner code maintenance.

6. Resource Efficiency :

- Requires standard JVM runtime environments with minimal memory allocation (< 15 MB heap).
- No heavy third-party dependencies or external JAR files.

SYSTEM ARCHITECTURE :

1. User Interface Layer :

- Provides a **menu-driven console interface**.
- Accepts user input for selecting operations (1-9).
- Displays prompts and results in a clear format.

2. Application Logic Layer :

- Contains **modular methods** for each functionality:
 - scientificFunctions() → Handles trigonometric, logarithmic, exponential, and hyperbolic operations.
 - matrixOperations() → Performs matrix addition and multiplication.

VITYARTHI – PROJECT

- `quadraticSolver()` → Computes real or complex roots of quadratic equations.
- `linearSolver2x2()` → Solves simultaneous linear equations using determinants.
- `statistics()` → Calculates mean, median, and standard deviation.
- `primeNumbers()` → Checks primality of integers.
- `factorial()` → Computes factorial of a number.
- `power()` → Calculates exponentiation.

3. Exception Handling Layer :

- A global try-catch block in the `main()` loop ensures robustness.
- Handles:
 - `InputMismatchException` → Invalid input types.
 - `ArithmeticException` → Mathematical errors (e.g., division by zero).
 - `Generic Exception` → Any unexpected runtime errors.
- Provides user-friendly error messages without crashing the program.

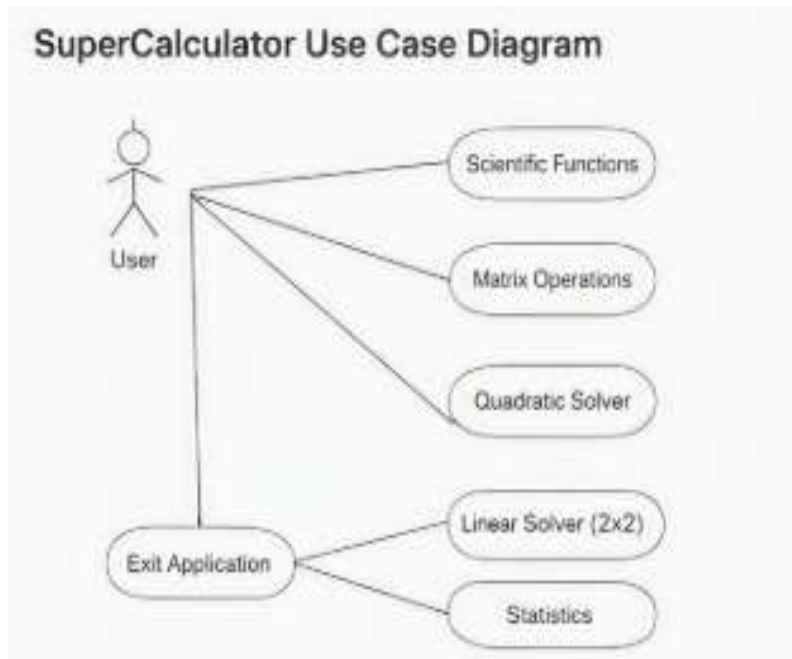
4. Data Layer :

- Uses **Java Scanner** for input handling.
- Stores values in arrays (for statistics) and matrices (for matrix operations).
- Ensures proper validation before performing computations.

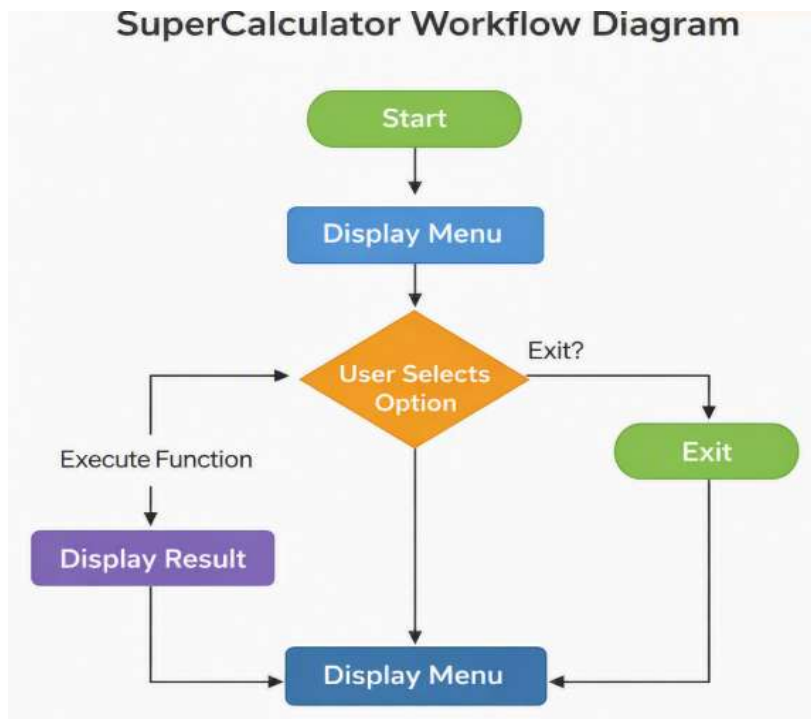
VITYARTHI – PROJECT

DESIGN DIAGRAMS :

1. Use Case Daigram:

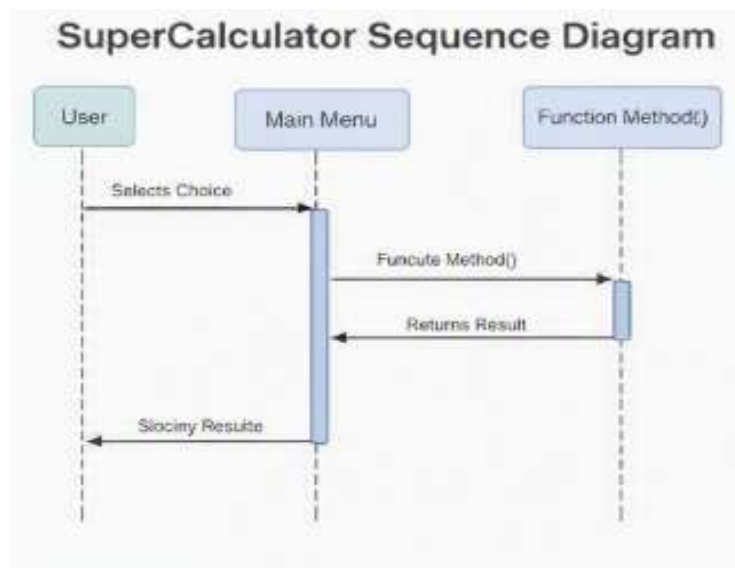


2. Work Flow Daigram:

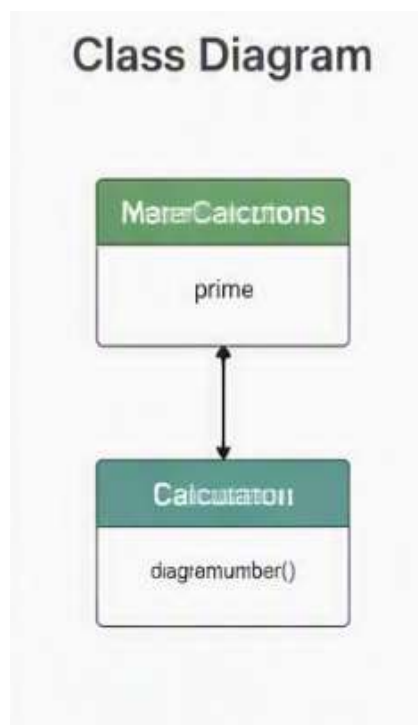


VITYARTHI – PROJECT

3. Sequence Daigram:



4. Class Daigram:



VITYARTHI – PROJECT

DESIGN DECISIONS & RATIONALE :

1. Programming Language: Java :

- **Decision:** The project is implemented in Java.
- **Rationale:**
 - Java provides strong support for object-oriented programming.
 - It offers built-in mathematical libraries (Math class) for scientific and trigonometric operations.
 - Platform independence (via JVM) ensures the program runs on any system without modification.

2. Architecture Style: Modular Design :

- **Decision:** Each calculator function is implemented as a separate method.
- **Rationale:**
 - Promotes code readability and maintainability.
 - Allows easy debugging and future extension (e.g., adding new operations).
 - Reduces coupling between components.

3. User Interface: Console-Based Menu :

- **Decision:** A text-based menu guides the user through available operations.
- **Rationale:**
 - Simple and lightweight — no GUI dependencies.
 - Ideal for academic and command-line environments.
 - Encourages focus on logic rather than interface complexity.

4. Input Handling: Scanner Class :

- **Decision:** `java.util.Scanner` is used for user input.
- **Rationale:**
 - Provides easy parsing of numeric and string data.
 - Simplifies interaction with the console.

VITYARTHI – PROJECT

5. Exception Handling: Global Try-Catch :

- **Decision:** A single global try-catch block wraps the main loop.
- **Rationale:**
 - Prevents program termination on invalid input.
 - Ensures smooth user experience with friendly error messages.
 - Simplifies error management across all modules.

6. Scalability & Extensibility :

- **Decision:** The design supports adding new modules (e.g., graph plotting, complex number operations).
- **Rationale:**
 - Modular structure allows future enhancements without altering existing logic.

IMPLEMENTATION DETAILS :

- Language: Java (JDK 17+)
- Development Environment: Visual Studio Code
- Source File: SuperCalculator.java
- Primary Class: SuperCalculator

(i)Menu-Driven Interface:

Users select an operation from the displayed menu.

(ii)Input Handling:

The program prompts for required values
(numbers, matrices, datasets).

VITYARTHI – PROJECT

(iii)Computation:

Each method performs its respective calculation using Java's Math library or custom logic.

(iv)Formatted Output:

Results are displayed clearly, with support for real and complex numbers, matrix formatting, and statistical precision

VITYARTHI – PROJECT

TESTING APPROACH :

1. Testing Objectives:

- Validate the accuracy of mathematical computations.
- Ensure proper handling of user inputs and exceptions.
- Confirm that the menu-driven interface operates smoothly.
- Verify that invalid inputs do not crash the program.

2. Testing Techniques :

- **Black Box Testing:** Focuses on input-output behavior without inspecting internal code logic.
- **White Box Testing:** Ensures logical correctness of loops, conditions, and calculations.
- **Boundary Testing:** Tests edge cases (e.g., factorial of 0, prime check for 1, matrix size limits).
- **Error Handling Testing:** Verifies that exceptions are caught and handled gracefully.

3. Expected Outcome:

- All modules should execute correctly, handle invalid inputs gracefully, and display accurate results. The system should remain stable across multiple iterations without memory leaks or crashes.

VITYARTHI – PROJECT

CHALLENGES FACED :

Input Handling :

- Managing invalid user inputs (e.g., entering text instead of numbers).
- Preventing the program from crashing when unexpected values were provided.

Mathematical Accuracy :

- Handling domain restrictions for trigonometric and inverse functions (e.g., $\text{asin}(x)$ and $\text{acos}(x)$ only valid for $|x| \leq 1$).
- Ensuring precision in floating-point calculations, especially for statistical functions.

Matrix Operations :

- Implementing matrix multiplication correctly with nested loops.
- Printing results in a readable row-column format instead of a single line.

Equation Solvers :

- Handling cases where the determinant in the linear solver was zero (no unique solution).
- Managing complex roots in the quadratic solver.

Exception Handling

- Designing a global try-catch block to handle all runtime errors gracefully.
- Clearing the scanner buffer after invalid inputs to avoid infinite loops.

VITYARTHI – PROJECT

OUTPUT:

```
PS C:\Users\dilip\OneDrive\Desktop\Java> & 'C:\Program Files\Java\jdk-24\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" -cp "C:\Users\dilip\AppData\Local\Temp\Code\User\workspacestorage\1ae97417c1e986a27bf25c75fba95c54\src\dhut.java\jdk_src\java_24\bin\bin" SuperCalculator

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 1
Functions: log, log10, exp, sin, cos, tan, asin, acos, atan, sinh, cosh, tanh
Enter a number: 90
log(x) = 4.499809670330265
log10(x) = 1.954242589439325
exp(x) = 1.2204032943178408E39
sin(x°) = 1.0
cos(x°) = 6.123233995736766E-17
tan(x°) = 1.633123935319517E16
asin(x) = NaN
acos(x) = NaN
atan(x) = 89.36340642403653
sinh(x) = 6.102016471589204E38
cosh(x) = 6.102016471589204E38
tanh(x) = 1.0

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 2
Enter size of matrix (n): 2
Enter matrix A:
Enter element A[0][0]: 1
```

```
Enter matrix A:
Enter element A[0][0]: 1
Enter element A[0][1]: 2
Enter element A[1][0]: 3
Enter element A[1][1]: 4
Enter matrix B:
Enter element B[0][0]: 3
Enter element B[0][1]: 5
Enter element B[1][0]: 6
Enter element B[1][1]: 8
A*B: 4 7 9 12
A*B: 15 21 33 47
--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 3
Enter coefficients a: 25
Enter coefficients b: 19
Enter coefficients c: 11
Complex roots

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 4
Equation format: a1x + b1y = c1 ; a2x + b2y = c2
Enter a1: 1
Enter b1: 2
Enter c1: 3
Enter a2: 6
```


VITYARTHI – PROJECT

```
Equation format: a1x + b1y = c1 ; a2x + b2y = c2
Enter a1: 1
Enter b1: 2
Enter c1: 3
Enter a2: 6
Enter b2: 4
Enter c2: 1
Solution: x=-1.25, y=2.125

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 5
Enter number of elements: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Mean = 3.0
Median = 3.0
Standard Deviation = 1.4142135623730951

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 6
Enter base: 2
Enter exponent: 10
Power(2.0*10.0) = 1024.0

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 7
Enter a number: 25
25 is Not Prime

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 8
Enter a number: 5
Factorial(5) = 120

--- Super Calculator ---
1. Scientific Functions
2. Matrix Operations
3. Quadratic Solver
4. Linear Solver (2x2)
5. Statistics
6. power
7. Prime Numbers
8. Factorial
9. Exit
Enter the choice: 9
Thanks for using the calculator!
PS C:\Users\dilip\OneDrive\Desktop\java>
```

VITYARTHI – PROJECT

LEARNINGS AND KEY TAKEAWAYS :

1. Mathematical Computation with Libraries :

- Gained experience using Java's Math class for scientific, trigonometric, and exponential functions.

2. Problem-Solving with Algorithms :

- Implemented matrix addition and multiplication using nested loops.
- Applied determinant-based logic for solving linear equations.
- Designed algorithms for prime number checking and factorial computation.

3. Exception Handling and Robustness :

- Learned how to use global try-catch blocks to prevent program crashes.
- Improved user experience by handling invalid inputs gracefully.

4. User Interaction Design :

- Designed a menu-driven console interface that is simple, intuitive, and easy to navigate.
- Ensured clear prompts and outputs for better usability.

5. Testing and Validation :

- Practiced black-box and boundary testing to validate correctness.
- Verified outputs against manual calculations to ensure accuracy.

VITYARTHI – PROJECT

6. Software Engineering Practices :

- Applied design diagrams (Use Case, Workflow, Sequence, Class, ER) to visualize system architecture.
- Understood the importance of documentation in communicating design decisions and rationale.

FUTURE ENHANCEMENTS :

Graphical User Interface (GUI) :

- Replace the console menu with a GUI using JavaFX or Swing.
- Provide buttons, text fields, and result panels for a more user-friendly experience.

Persistent Storage :

- Add database integration (e.g., MySQL or SQLite) to store past calculations.
- Allow users to retrieve history and reuse previous inputs.

Advanced Mathematical Features :

- Support for complex numbers and operations.
- Matrix inversion, determinant, and eigenvalue calculations.
- Additional statistical measures (variance, correlation, regression).

Modular Expansion :

- Introduce new modules like calculus (derivatives, integrals), probability distributions, and number theory.
- Enable plug-and-play design so new functions can be added easily.`

VITYARTHI – PROJECT

REFERENCES :

VITyarthi Project Guide – *Build Your Own Project* (PDF)

Oracle Java Documentation – *Java SE API Specification*

Stack Overflow – *Best Practices for Exception Handling in Java*

GitHub Repository – *Java_Vityarthi Project*

THANK YOU